

TypeScript aurreratua – Ariketa multzoa

Ariketa multzo hau TypeScript oinarrizko ariketak lehenago amaitu dituzten ikasleentzat proposatzen da. Kontzeptu berriak sartzen ditu oraingoz landu ez direnak; horregatik, ariketa bakoitzaren aurretik kontzeptu horren azalpen labur bat eta adibide bat aurkituko dituzu.

1. Enkapsulazioa: public, private, protected eta readonly

UD 1

Iraupena: 30 min

Kontzeptu berria: Enkapsulazioa

TypeScriptek hiru sarbide-eragile eskaintzen ditu klaseetako propietate eta metodoetarako: public (lehenetsia, edonondik atzitu daiteke), private (klasearen barruan bakarrik atzitu daiteke) eta protected (klasean eta bere azpiklaseetan atzitu daiteke, baina kanpotik ez).

Gainera, readonly hitza erabil daiteke propietate bat behin bakarrik (eraikitzailean, normalean) balioztatzeko, gero aldatu ezin dena. Propietate pribatuak modu kontrolatuan irakurri edo aldatzeko, get eta set metodo bereziak (accessor-ak) erabil daitezke.

```
class Kontagailua {  
    private balioa: number = 0;  
  
    get uneko(): number {  
        return this.balioa;  
    }  
  
    handitu(): void {  
        this.balioa++;  
    }  
}
```

Helburua: enkapsulazioa aplikatzea, klase baten datuak kanpotik babesteko eta modu kontrolatuan bakarrik aldatu ahal izateko.

1. Sortu KontuBankarioa klase bat honako ezaugarriekin:
 - titularra: public eta readonly (eraikitzailean bakarrik ezarri behar da).
 - saldoa: private.
2. Gehitu sartu(kopurua: number): void metodoa, saldoa handitzen duena. Kopurua positiboa ez bada, erakutsi errore-mezu bat kotsolan eta ez aldatu saldoa.
3. Gehitu atera(kopurua: number): void metodoa, saldotik kentzen duena. Saldoa nahikoa ez bada, erakutsi errore-mezu bat kotsolan eta ez aldatu saldoa.
4. Gehitu unekoSaldoa izeneko getter bat; saldoa kanpotik irakurtzeko modu bakarra izan behar da.
5. Sortu kontu bat, egin hainbat eragiketa (sartu, atera, saldo faltarekin saiatu) eta erakutsi emaitzak kotsolan.

6. Erronka gehigarria: saiatu kanpotik zuzenean kontu.saldoa atzitzen edo aldatzen, eta azaldu iruzkin gisa zergatik ematen duen TypeScriptek errorea.

2. Klase abstraktuak eta interfazeen implementazioa

UD 1

Iraupena: 30 min

Kontzeptu berria: klase abstraktuak eta implements

Klase abstraktu bat (abstract class) ezin da zuzenean instantziatu; oinarri gisa balio du beste klaseentzat. Metodo abstraktuek (abstract hitzarekin markatuek) sinadura bakarrik dute, eta azpiklase bakoitzak bere implementazio propioa eman behar dio.

Bestalde, klase batek interfaze bat implementa dezake implements hitz-gakoa erabiliz; horrela, klaseak interfazeen zehaztutako propietate eta metodo guztiak izan behar ditu derrigorrez.

```
abstract class Forma {
    abstract azalera(): number;

    deskribapena(): string {
        return `Azalera: ${this.azalera()}`;
    }
}
```

Helburua: klase abstraktuak eta interfazeen implementazioa erabiltzea, oinarrizko egitura partekatu bat definitzeko eta azpiklase bakoitzak bere xehetasunak nola betetzen dituen ulertzeko.

1. Sortu Forma klase abstraktu bat, azalera(): number metodo abstraktuarekin eta deskribapena(): string metodo arruntarekin (barruan azalera() erabiliz).
2. Sortu Zirkulua klasea, Forma-tik heredatzen duena, erradioa propietatearekin eta azalera() metodoa inplementatuta ($\pi * \text{erradioa}^2$).
3. Sortu Laukizuzena klasea, Forma-tik heredatzen duena, zabalera eta altuera propietateekin eta azalera() metodoa inplementatuta.
4. Sortu bi objektu (Zirkulua bat eta Laukizuzena bat), Forma[] motako array batean bildu, eta erakutsi kontsolan bakoitzaren deskribapena().
5. Sortu Marrazgarria interfaze bat, marraztu(): void metodoarekin, eta egin Zirkulua klaseak interfaze hori ere inplementa dezan (implements Marrazgarria), marraztu() metodoa gehituz.

Ariketa osagarriak

3. Enumerazioak eta literal motak

UD 1

Iraupena: 30 min

Kontzeptu berria: enum eta literal motak

enum bat izen esanguratsuak dituzten konstante multzo bat definitzeko erabiltzen da. Kodea irakurgarriagoa egiten du, testu edo zenbaki 'magikoak' erabili beharrean.

Beste aukera bat literal motak konbinatzea da (string literal types): balio zehatz batzuk bakarrik onartzen dituen mota bat sortzen da, string generiko baten ordeztzeko.

```
enum Egoera {
    Aktibo,
    EzAktibo,
    Ezabatuta
}

type Tamaina = 'S' | 'M' | 'L' | 'XL';
```

Helburua: enum eta literal motak erabiltzea, aldagai batek onar ditzakeen balioak modu argi eta seguruan mugatzeko.

1. Sortu Egoera enum bat honako balioekin: Aktibo, EzAktibo, Ezabatuta.
2. Sortu Erabiltzailea interfaze bat izena: string eta egoera: Egoera propietateekin.
3. Sortu gutxienez 4 erabiltzaile dituen array bat, egoera ezberdinekin.
4. Idatzi aktiboakDira(erabiltzaileak: Erabiltzailea[]): Erabiltzailea[] funtzio bat, Egoera.Aktibo dutenak bakarrik itzultzen dituen, filter erabiliz.
5. Sortu Tamaina literal mota bat ('S' | 'M' | 'L' | 'XL') eta idatzi stockKontrola(tamaina: Tamaina, kopurua: number): string funtzio bat, tamaina eta kopurua deskribatzen dituen testu bat itzultzen duena.
6. Deitu bi funtzioei eta erakutsi emaitzak kontsolan.

4. Motak zehaztea (type narrowing)

UD 1

Iraupena: 45 min

Kontzeptu berria: type narrowing

Union motekin lan egitean (adibidez, `string | number`), askotan komeni da exekuzio-unean zehaztea zein motatakoa den benetan balio bat; horri type narrowing deitzen zaio. Teknika ohikoenak: `typeof` (oinarrizko motak egiaztatzeko), `instanceof` (klase baten instantzia den egiaztatzeko), in eragilea (objektu batek propietate jakin bat duen egiaztatzeko) eta erabiltzaileak definitutako type guard funtzioak, parametroa is Mota motako itzulera dutenak.

```
function isEnpresa(x: Bezeroa | Enpresa): x is Enpresa {
    return 'IFZ' in x;
}
```

Helburua: union motekin lan egitea, exekuzio-unean balio baten benetako mota zehaztuz, teknika ezberdinak erabiliz.

1. Idatzi zenbakiaEdoTestua(balioa: `number | string`): `string` funtzio bat, `typeof` erabiliz: balioa zenbakia bada 'Zenbakia: X' eta testua bada 'Testua: X' itzuli behar du.
2. Sortu Katua eta Txakurra klaseak, bakoitzak bere metodo propioarekin (`miaukaEgin()` eta `zaunkaEgin()`, hurrenez hurren).

3. Idatzi `soinuaEgin`(animalia: Katua | Txakurra): void funtzio bat, `instanceof` erabiliz, animalia-aren motaren arabera metodoa deitzen duena.
4. Sortu `Bezeroa` (izena, helbidea) eta `Enpresa` (izena, IFZ) interfazeak, eta idatzi type guard pertsonalizatu bat: `isEnpresa(sarrera: Bezeroa | Enpresa): sarrera is Enpresa`, in eragilea erabiliz.
5. Erabili type guard hori identifikatu(sarrera: Bezeroa | Enpresa): string funtzio batean, sarrera-ren motaren arabera mezu ezberdin bat itzultzeko.

5. Ariketa integratzaile aurreratua: utility types eta async/await

UD 1

Iraupena: 1 ordu

Kontzeptu berria: utility types eta async/await

TypeScriptek utility type izeneko mota lagungarri batzuk eskaintzen ditu, dagoeneko dagoen mota batetik beste mota berri bat sortzeko: `Partial<T>` (T-ren propietate guztiak aukerako bihurtzen ditu), `Readonly<T>` (guztiak readonly bihurtzen ditu), `Pick<T, K>` (K propietateak bakarrik hautatzen ditu) eta `Omit<T, K>` (K propietateak kentzen ditu).

Bestalde, `async/await` sintaxia eragiketa asinkronoekin lan egiteko erabiltzen da (adibidez, sarearen bidez datuak eskuratzean), `Promise` batekin batera. Funtzio bat `async` gisa markatzen bada, barruan `await` erabil daiteke, `Promise` bat ebatzi arte kodearen exekuzioa itxaroteko.

```
async function lortuDatuak(): Promise<string> {
    return new Promise((ebatzi) => {
        setTimeout(() => ebatzi('datuak eskuratuta'), 1000);
    });
}

async function main() {
    const emaitza = await lortuDatuak();
    console.log(emaitza);
}
```

Helburua: ikasitako kontzeptu guztiak (generikoak, klaseak) utility type berriekin eta programazio asinkronoarekin konbinatzea, egoera errealistagoa den mini-proiektu baten bidez.

1. Sortu Produktua interfaze bat (id: number, izena: string, prezioa: number, stock: number).
2. Sortu eguneratuProduktua(produktua: Produktua, aldaketak: Partial<Produktua>): Produktua funtzio bat, aldaketak jasotzen dituena eta produktu eguneratu bat itzultzen duena (jatorrizko produktua aldatu gabe, spread operatzailea erabiliz).
3. Sortu ProduktuLaburpena mota bat Pick<Produktua, 'izena' | 'prezioa'> erabiliz, eta idatzi produktu-zerrenda batetik laburpen-zerrenda bat sortzen duen funtzio bat (map erabiliz).
4. Sortu lortuProduktuak(): Promise<Produktua[]> funtzio async bat, setTimeout baten bidez sareko dei bat simulatzen duena (adibidez, segundo bat geroago produktu-array bat ebazten duena).
5. Idatzi main() izeneko funtzio async bat, await erabiliz lortuProduktuak() deitzen duena, eta emaitza kotsolan erakusten duena (produktu bakoitzaren laburpena barne, 3. puntuko funtzioa erabiliz).
6. Erronka gehigarria: aplikatu aurreko ariketetako Biltegia<T> klase generikoa Produktua motarekin, eta gehitu lortuProduktuak() funtzioak itzultzen dituen produktuak biltegira, async/await erabiliz.